

Chapter 5: Flow Control

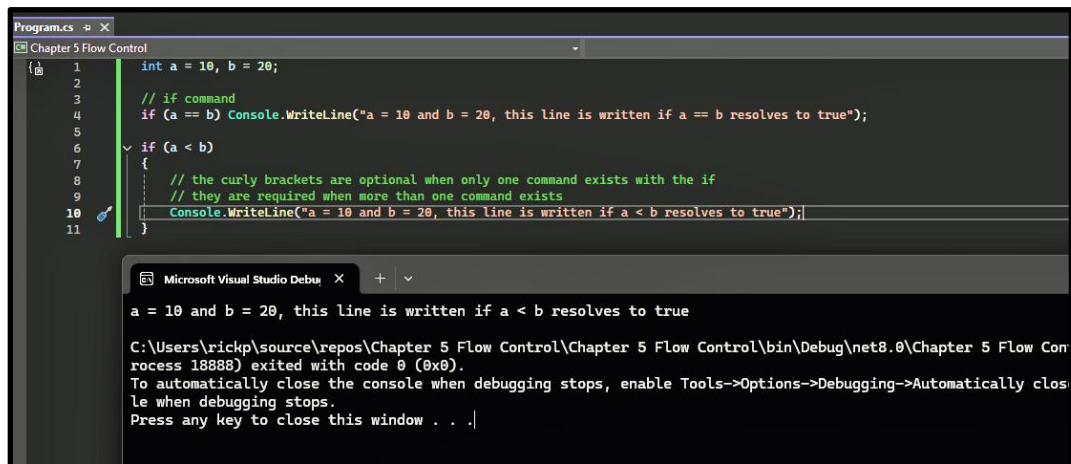
Flow control refers to the order in which commands and/or subprograms are executed within an application. The typical situation is something like “execute this section of code if some conditional is true” or “execute this subprogram a specified number of times” or “execute this section of code once for each member of a set”. Super important stuff because elegant flow control is a cornerstone to good programming.

Okay, let’s jump to the inside and take a look at some flow control examples.

Please begin by creating a new console application project named chapter 5 flow control.

We will start with a simple if statement. (Figure 5.1)

Figure 5.1: if



```
Program.cs
Chapter 5 Flow Control
1  int a = 10, b = 20;
2
3  // if command
4  if (a == b) Console.WriteLine("a = 10 and b = 20, this line is written if a == b resolves to true");
5
6  if (a < b)
7  {
8      // the curly brackets are optional when only one command exists with the if
9      // they are required when more than one command exists
10     Console.WriteLine("a = 10 and b = 20, this line is written if a < b resolves to true");
11 }
```

Microsoft Visual Studio Debug Console

a = 10 and b = 20, this line is written if a < b resolves to true

C:\Users\rickp\source\repos\Chapter 5 Flow Control\Chapter 5 Flow Control\bin\Debug\net8.0\Chapter 5 Flow Control.exe exited with code 0 (0x0).

To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close console when debugging stops.

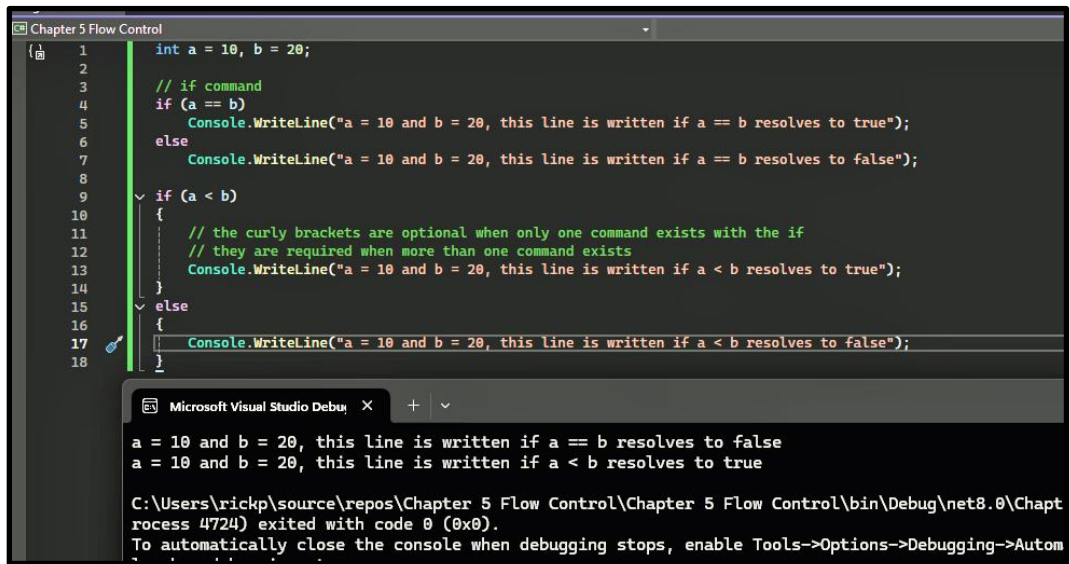
Press any key to close this window . . .

So, what did we learn?

- First, that the equality test in C# is represented by a pair of equal signs.
- Next, that an IF command tests a conditional and if that conditional resolves to true, the subsequent command or commands are executed. If the conditional resolves to false, the subsequent command or commands are not executed. In our example here the first Console.WriteLine command is not executed because variable a does not equal variable b. The second Console.WriteLine command is executed because variable a is less than variable b.
- Finally, we learned that the curly brackets are optional if the executed code block includes only a single command but are required if the executed code block contains more than one command. Net, the curly brackets delimit the to-be-executed code.

Next let's update our code to include the else keyword. (Figure 5.2)

Figure 5.2: if else Syntax



```
Chapter 5 Flow Control
1  int a = 10, b = 20;
2
3  // if command
4  if (a == b)
5      Console.WriteLine("a = 10 and b = 20, this line is written if a == b resolves to true");
6  else
7      Console.WriteLine("a = 10 and b = 20, this line is written if a == b resolves to false");
8
9  if (a < b)
10 {
11     // the curly brackets are optional when only one command exists with the if
12     // they are required when more than one command exists
13     Console.WriteLine("a = 10 and b = 20, this line is written if a < b resolves to true");
14 }
15 else
16 {
17     Console.WriteLine("a = 10 and b = 20, this line is written if a < b resolves to false");
18 }
```

Microsoft Visual Studio Debug Console

```
a = 10 and b = 20, this line is written if a == b resolves to false
a = 10 and b = 20, this line is written if a < b resolves to true

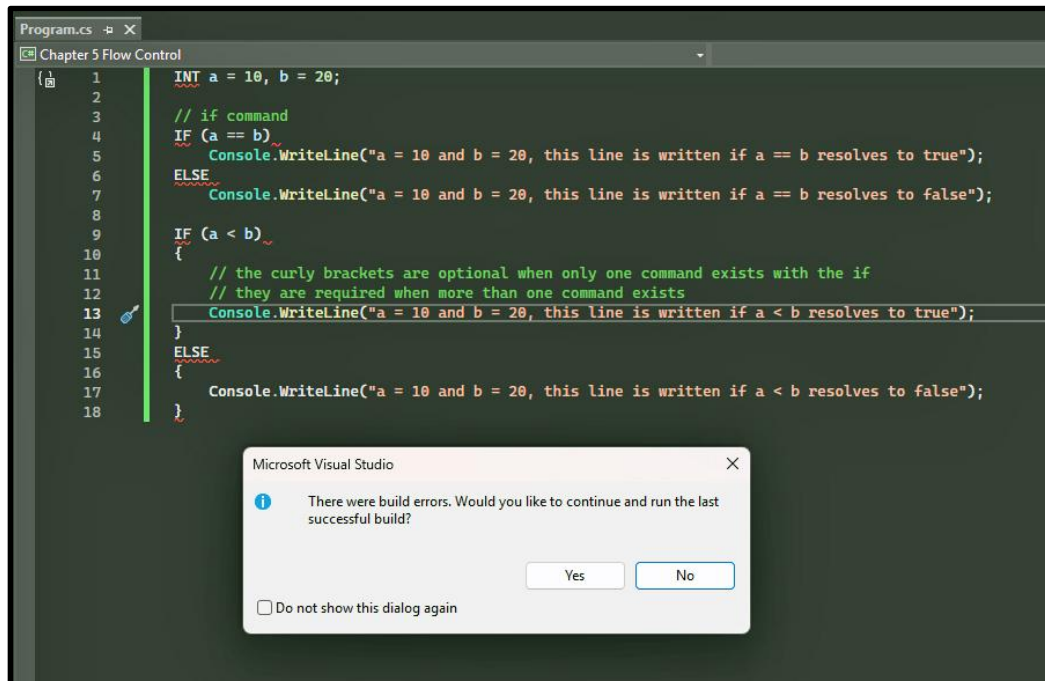
C:\Users\rickp\source\repos\Chapter 5 Flow Control\Chapter 5 Flow Control\bin\Debug\net8.0\Chapt
rocess 4724) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Autom
```

So, what did we learn from this example?

- That the else code block is executed when the original if conditional resolves to false.

Now let's change our code to have all the keywords in uppercase. (Figure 5.3)

Figure 5.3: Keywords Are All Lowercase

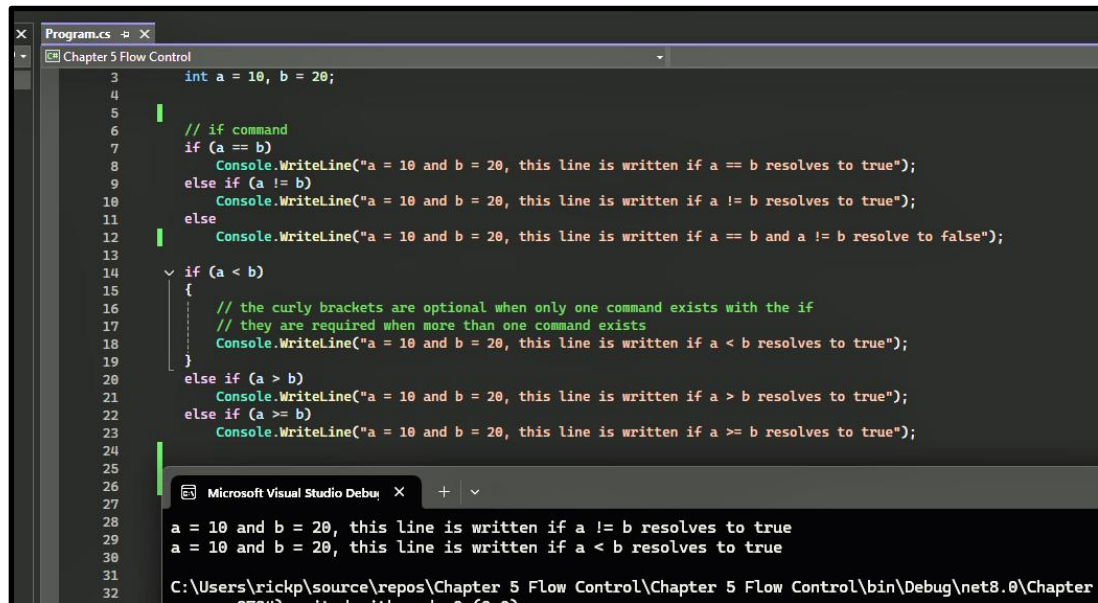


What did we learn from this example?

- That the C# compiler did not recognize the INT, IF and ELSE keywords when they were in uppercase. C# has 79 keywords, each is lowercase.
- We also learned that intellisense recognized the problem and underlined the improperly formatted keywords in red.
- Finally, please note the formatting of the Console.WriteLine method. The use of a capital letter at the beginning of each word is known as PascalCase. Microsoft uses this format in all it's preprogrammed methods. By the way, this dot notation command says that we need to go to the Console class and once there execute the WriteLine method.

In our next example we include the else if syntax. (Figure 5.4) In the if, else if, else syntax the if conditional is evaluated first. If it resolves to true, the code block associated with it is executed and the remaining else if and else conditionals are ignored. When the if conditional resolves to false the flow control moves to the first else if conditional. If that conditional resolves to true, then the code block associated with it is executed and the remaining code is ignored. Any number of else if conditionals may exist and they are evaluated top to bottom. Only the code block associated with the first one which resolves to true is executed. If none of the if or else if conditionals resolves to true, then the else code block is executed. The else block is optional. I have included it here in our first command block but excluded it in our second.

Figure 5.4: else if



```
Program.cs
Chapter 5 Flow Control
3 int a = 10, b = 20;
4
5
6 // if command
7 if (a == b)
8     Console.WriteLine("a = 10 and b = 20, this line is written if a == b resolves to true");
9 else if (a != b)
10    Console.WriteLine("a = 10 and b = 20, this line is written if a != b resolves to true");
11 else
12    Console.WriteLine("a = 10 and b = 20, this line is written if a == b and a != b resolve to false");
13
14 if (a < b)
15 {
16     // the curly brackets are optional when only one command exists with the if
17     // they are required when more than one command exists
18     Console.WriteLine("a = 10 and b = 20, this line is written if a < b resolves to true");
19 }
20 else if (a > b)
21     Console.WriteLine("a = 10 and b = 20, this line is written if a > b resolves to true");
22 else if (a >= b)
23     Console.WriteLine("a = 10 and b = 20, this line is written if a >= b resolves to true");
24
25
26
27
28
29
30
31
32
```

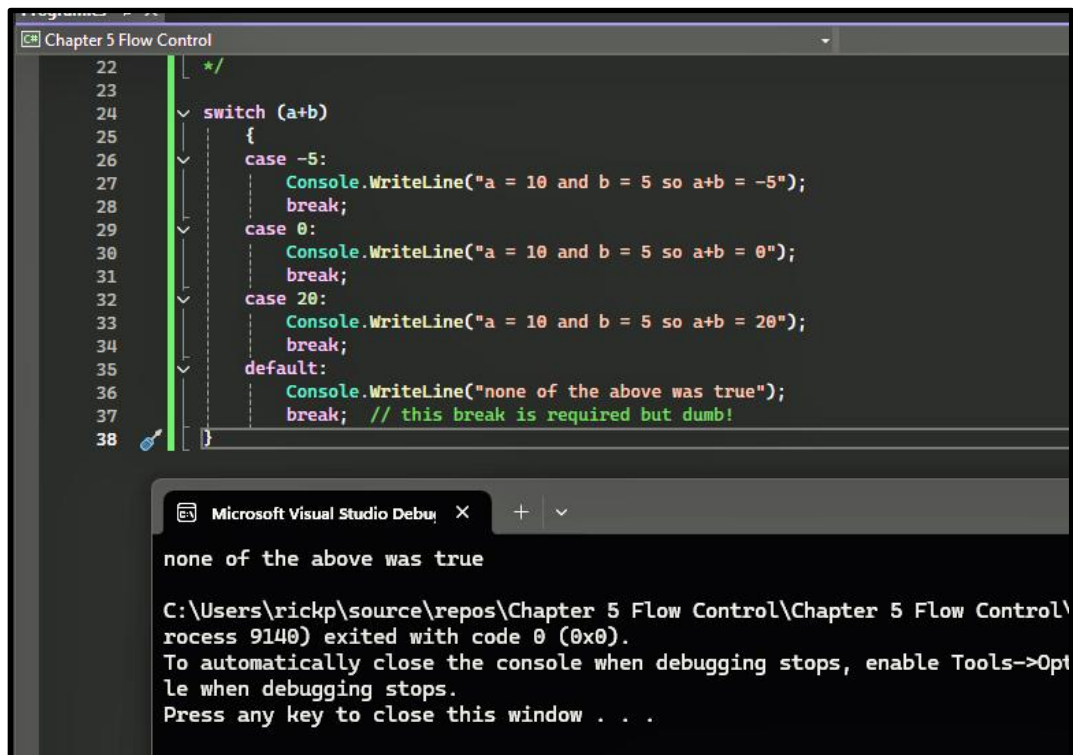
Microsoft Visual Studio Debug

```
a = 10 and b = 20, this line is written if a != b resolves to true
a = 10 and b = 20, this line is written if a < b resolves to true
C:\Users\rickp\source\repos\Chapter 5 Flow Control\Chapter 5 Flow Control\bin\Debug\net8.0\Chapter
```

We move next to the switch/case flow control syntax. (Figure 5.5) I think the code here is pretty self-explanatory other than the break keyword. The break keyword simply says stop what you are doing within this block of code and jump to the bottom of the parent command. In our example, that parent command is the switch keyword. By the way, I coded for some 20+ years in C and C# and never used or even saw the switch command. Everybody uses the if syntax instead. However, I know some of you will be taking certification exams as associated with this content, so I include the switch/case command just in case this syntax lives on one of those tests. As an aside, I believe my comment regarding the required break command with the default keyword is accurate. Think it through, the only time that should be required is when the default keyword precedes one of the case options and that would be just too weird. Net, stick with the if syntax.

By the way, I have commented out all my if syntax at this point to keep the output clean. I did however leave in my a and b variable declarations and assignments.

Figure 5.5: switch/case



The screenshot shows a Visual Studio IDE window titled "Chapter 5 Flow Control". The code editor displays a C# switch/case statement. The code is as follows:

```
22  */
23
24  switch (a+b)
25  {
26  case -5:
27      Console.WriteLine("a = 10 and b = 5 so a+b = -5");
28      break;
29  case 0:
30      Console.WriteLine("a = 10 and b = 5 so a+b = 0");
31      break;
32  case 20:
33      Console.WriteLine("a = 10 and b = 5 so a+b = 20");
34      break;
35  default:
36      Console.WriteLine("none of the above was true");
37      break; // this break is required but dumb!
38  }
```

Below the code editor, the "Microsoft Visual Studio Debug Console" is open, showing the output of the program:

```
none of the above was true

C:\Users\rickp\source\repos\Chapter 5 Flow Control\Chapter 5 Flow Control\
rocess 9140) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Opt
le when debugging stops.
Press any key to close this window . . .
```

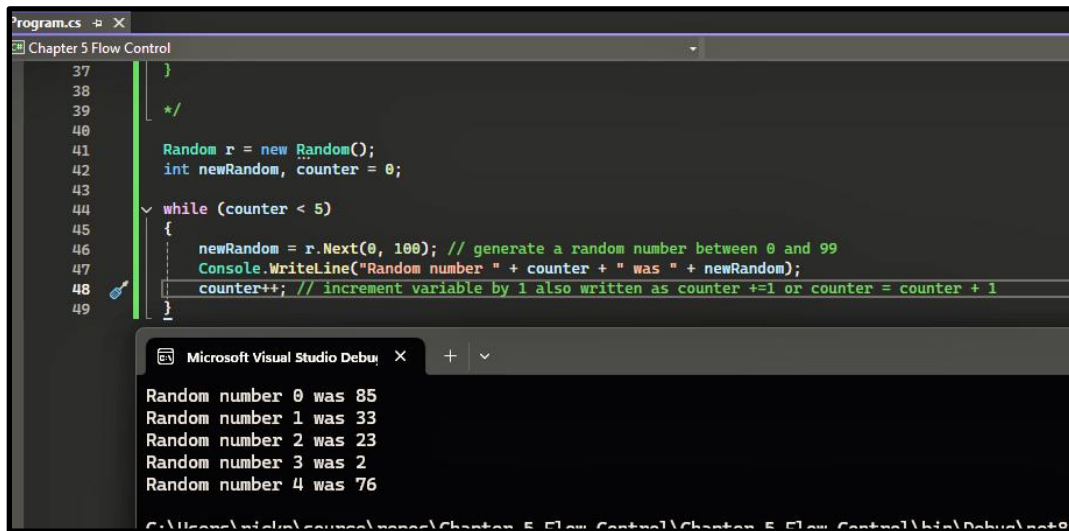
In our next example we introduce the while keyword. (Figure 5.6) The while syntax executes a block of code while a conditional resolves to true. In our example the block of code is executed while a variable named counter is less than 5.

What did we learn from this example?

- We learned how to use the while keyword to control the number of times a block of code is executed.
- We also learned about the preprogrammed class named Random. We will learn more about classes in chapters 7 and 8. For now just know that a class is built up from data type primitives or other classes and may have properties (the typical person class for example will have properties of first and last name, gender, email address, etc.) and methods (things they can do or execute, in the case of our Random class it has a method which can return a random number between two specified input parameters).
- We also learned two new ways to increment a variable by one. Of course we could have written "counter = counter + 1;" but we chose instead to write the most abbreviated version of that syntax as "counter++;". Please also note my comment which says there is even a third form of this syntax which reads "counter +=1;"

What do you suppose happens if you forget to increment the counter variable in this example? Yes of course, the program continues to execute indefinitely. This scenario is called an infinite loop and they are most definitely not cool. Easy enough to just shut down Visual Studio if you are working on your own box but if you are working on a big server farm or connected to a database it can be very embarrassing. Some systems administrator needs to physically shut down your process, not cool.

Figure 5.6: while



The screenshot shows a Visual Studio IDE with a C# file named 'Program.cs'. The code defines a 'while' loop that runs as long as 'counter' is less than 5. Inside the loop, a random number is generated and printed to the console, and the counter is incremented by 1. The output window shows five iterations of the loop, each printing a random number for counter values 0 through 4.

```
37 }
38
39 */
40
41 Random r = new Random();
42 int newRandom, counter = 0;
43
44 while (counter < 5)
45 {
46     newRandom = r.Next(0, 100); // generate a random number between 0 and 99
47     Console.WriteLine("Random number " + counter + " was " + newRandom);
48     counter++; // increment variable by 1 also written as counter +=1 or counter = counter + 1
49 }
```

Microsoft Visual Studio Debug Console Output:

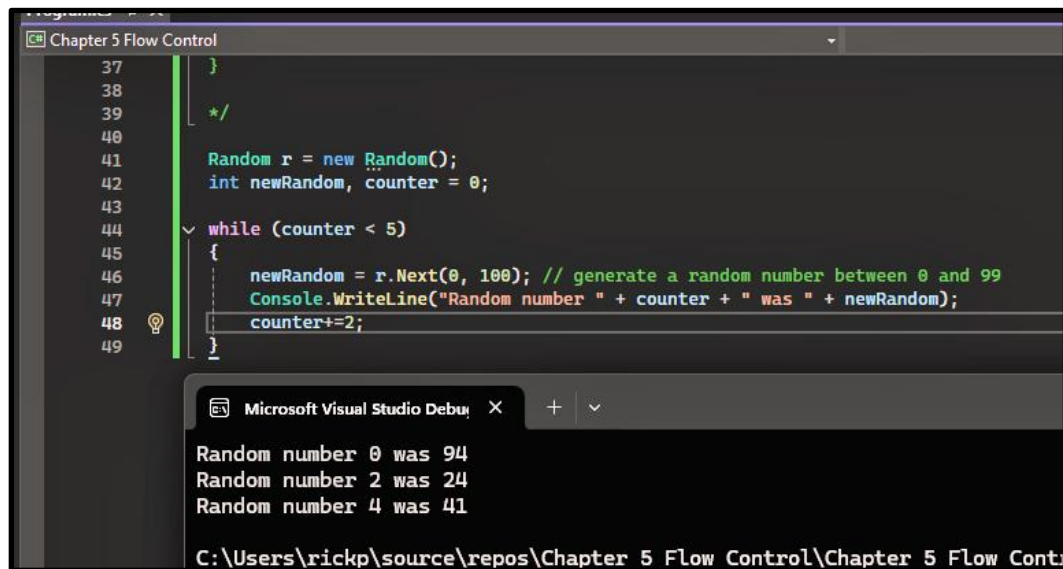
```
Random number 0 was 85
Random number 1 was 33
Random number 2 was 23
Random number 3 was 2
Random number 4 was 76
```

Okay, do me a favor and tell yourself without looking at the output what this next example should return. (Figure 5.7) If you said random numbers associated with the values 0, 2 and 4 then you are starting to think like a programmer...well done.

Just in case some of you were not able to properly anticipate the output, what did we learn from this example?

- That the variable `+= X` syntax takes variable and increments it by X with each pass of the loop. Okay, cool.

Figure 5.7: += syntax



```
37 }
38
39 */
40
41 Random r = new Random();
42 int newRandom, counter = 0;
43
44 while (counter < 5)
45 {
46     newRandom = r.Next(0, 100); // generate a random number between 0 and 99
47     Console.WriteLine("Random number " + counter + " was " + newRandom);
48     counter+=2;
49 }
```

Microsoft Visual Studio Debug Console

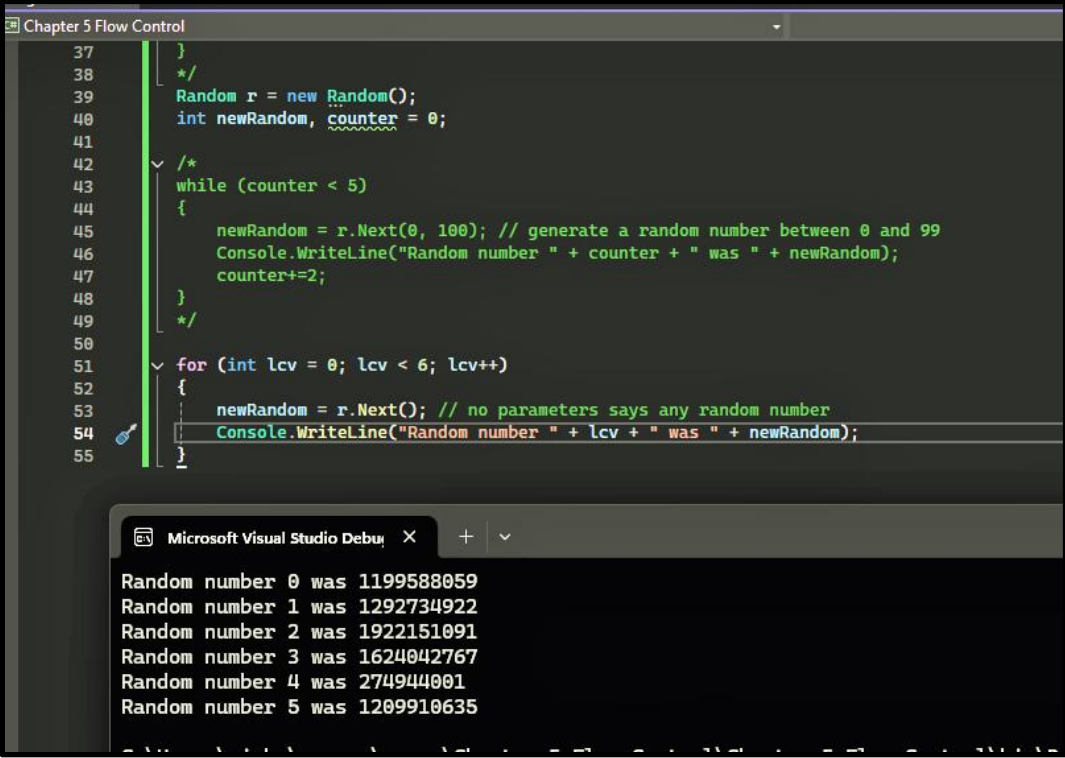
```
Random number 0 was 94
Random number 2 was 24
Random number 4 was 41
```

C:\Users\rickp\source\repos\Chapter 5 Flow Control\Chapter 5 Flow Control

In our next example we exchange the while keyword with the for keyword. (Figure 5.8) This guy requires some explanation.

- First, the for loop as shown here takes three parameters. The first in our example is an integer variable named lcv which in our case is initialized to zero. By the way, my choice here of a variable name was not by accident as this variable is generally known as the “loop control variable” such my use of the variable name lcv. The second parameter is a conditional. The loop is executed while that conditional resolves to true. The third, and final parameter, is used to change the loop control variable. In our case here, increment it by one with each execution of the code block.
- Next, check out what happens when you feed the Random class Next method zero parameters. Sure, it feels free to return any integer. By the way, most random number generators like this do modular arithmetic on the number of seconds past midnight. Just a fun fact to know and share at your next “geek party.”

Figure 5.8: for loop



The screenshot shows a Visual Studio editor window titled "Chapter 5 Flow Control". The code is in C# and includes a `while` loop and a `for` loop. The `for` loop is highlighted with a blue selection bar. Below the code, the Microsoft Visual Studio Debug Console shows the output of the program, displaying six random numbers generated by the `for` loop.

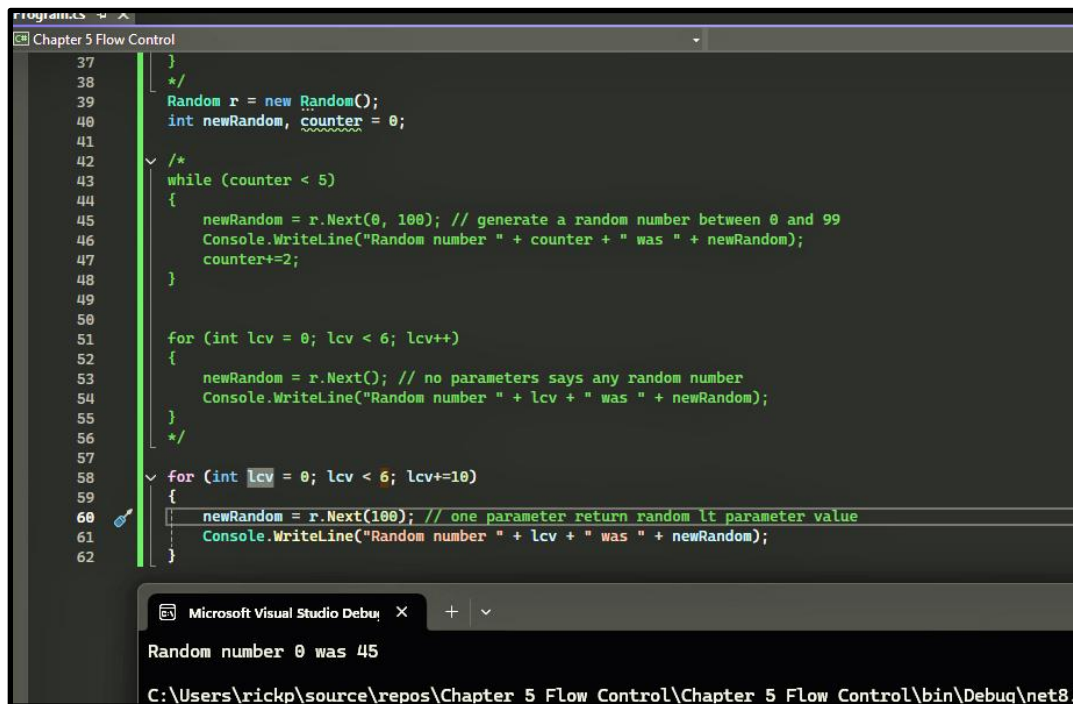
```
37     }  
38     */  
39     Random r = new Random();  
40     int newRandom, counter = 0;  
41  
42     /*  
43     while (counter < 5)  
44     {  
45         newRandom = r.Next(0, 100); // generate a random number between 0 and 99  
46         Console.WriteLine("Random number " + counter + " was " + newRandom);  
47         counter+=2;  
48     }  
49     */  
50  
51     for (int lcv = 0; lcv < 6; lcv++)  
52     {  
53         newRandom = r.Next(); // no parameters says any random number  
54         Console.WriteLine("Random number " + lcv + " was " + newRandom);  
55     }
```

Microsoft Visual Studio Debug Console Output:

```
Random number 0 was 1199588059  
Random number 1 was 1292734922  
Random number 2 was 1922151091  
Random number 3 was 1624042767  
Random number 4 was 274944001  
Random number 5 was 1209910635
```

In our next example (Figure 5.9) I used a random number seed, passed one parameter to the Next method of the Random class and changed the increment on my loop control variable to 10.

Figure 5.9: Updated for loop



```
37     }
38     */
39     Random r = new Random();
40     int newRandom, counter = 0;
41
42     /*
43     while (counter < 5)
44     {
45         newRandom = r.Next(0, 100); // generate a random number between 0 and 99
46         Console.WriteLine("Random number " + counter + " was " + newRandom);
47         counter+=2;
48     }
49
50     for (int lcv = 0; lcv < 6; lcv++)
51     {
52         newRandom = r.Next(); // no parameters says any random number
53         Console.WriteLine("Random number " + lcv + " was " + newRandom);
54     }
55     */
56
57     for (int lcv = 0; lcv < 6; lcv+=10)
58     {
59         newRandom = r.Next(100); // one parameter return random lt parameter value
60         Console.WriteLine("Random number " + lcv + " was " + newRandom);
61     }
62 }
```

Microsoft Visual Studio Debug Console

Random number 0 was 45

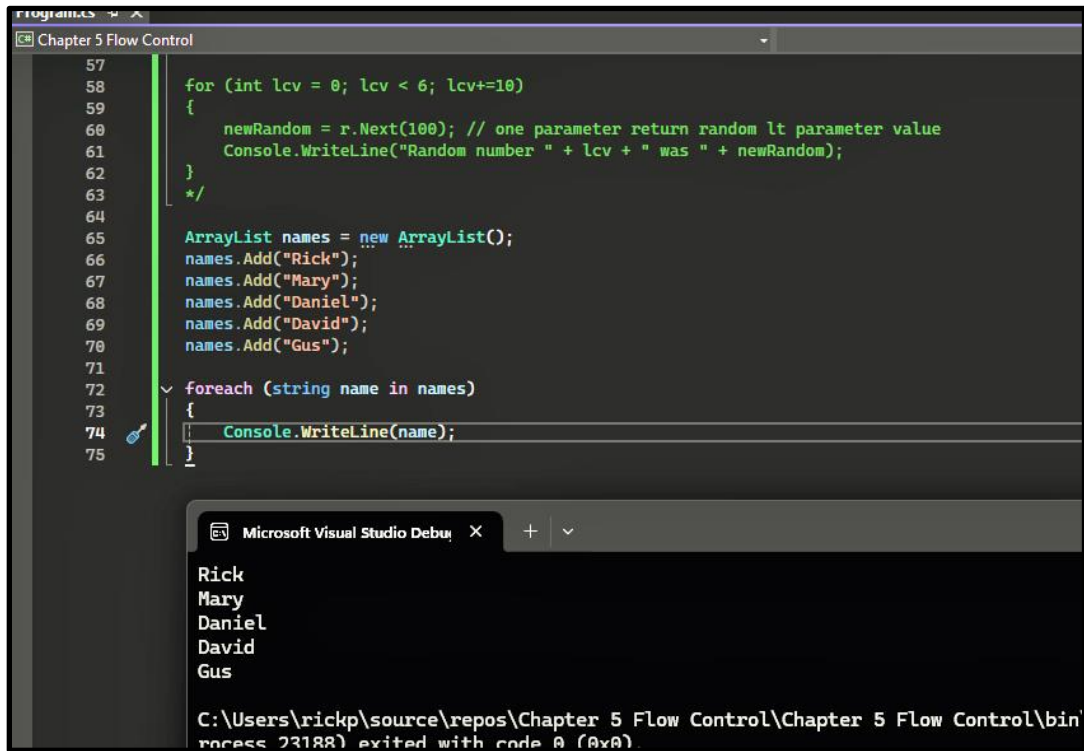
C:\Users\rickp\source\repos\Chapter 5 Flow Control\Chapter 5 Flow Control\bin\Debug\net8.

What did we learn?

- That we can seed the Random class Next method with a single parameter. If you repeatedly use the same seed, you will always get the same sequence of random numbers. Not important in this example but certainly very useful in other applications.
- That we can increment our loop control variable by any amount as long as the increment (or even decrement) is of the same data type as the loop control variable. Given this, do you suppose there are situations where your loop control variable is a double and the increment value is also a real number? Sure.

In our final flow control example we utilize an ArrayList class called names. (Figure 5.10) We then use the Add method of the ArrayList class to create five instances of the names class. Check out the foreach syntax. Do you suppose this is going to be valuable when we get to building classes and taking some actions on all the instances of that class? Sure.

Figure 5.10: foreach



```
57
58     for (int lcv = 0; lcv < 6; lcv+=10)
59     {
60         newRandom = r.Next(100); // one parameter return random lt parameter value
61         Console.WriteLine("Random number " + lcv + " was " + newRandom);
62     }
63     */
64
65     ArrayList names = new ArrayList();
66     names.Add("Rick");
67     names.Add("Mary");
68     names.Add("Daniel");
69     names.Add("David");
70     names.Add("Gus");
71
72     foreach (string name in names)
73     {
74         Console.WriteLine(name);
75     }
```

Microsoft Visual Studio Debug Console

```
Rick
Mary
Daniel
David
Gus

C:\Users\rickp\source\repos\Chapter 5 Flow Control\Chapter 5 Flow Control\bin\
Process 23188) exited with code 0 (0x0)
```

Well, that was a pretty heavy-duty chapter. We learned all about the various methods for controlling the order commands and subroutines are executed within an application.

In particular, we learned the following syntaxes:

- If else if else
- switch / case
- while
- for
- foreach

This is the most important stuff we have covered so far. Please make certain you have a full understanding of how each of our examples worked and why we got the results we did. There is no recipe for how to build a large integrated software solution. You take these basic building blocks and use them to create that solution. You of course will start out just maintaining and modifying someone else's code but after a few years you will be expected to create your own. To do so these basic building blocks have to become second nature so you can focus on the true design of your solution.

In our next lesson we will examine some of the older methods for storing data within an application, i.e. collections. Once we have those under our belts we can move on to classes.